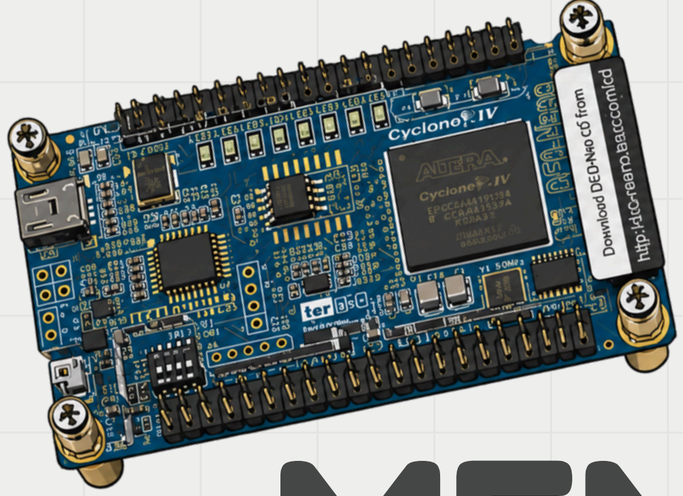
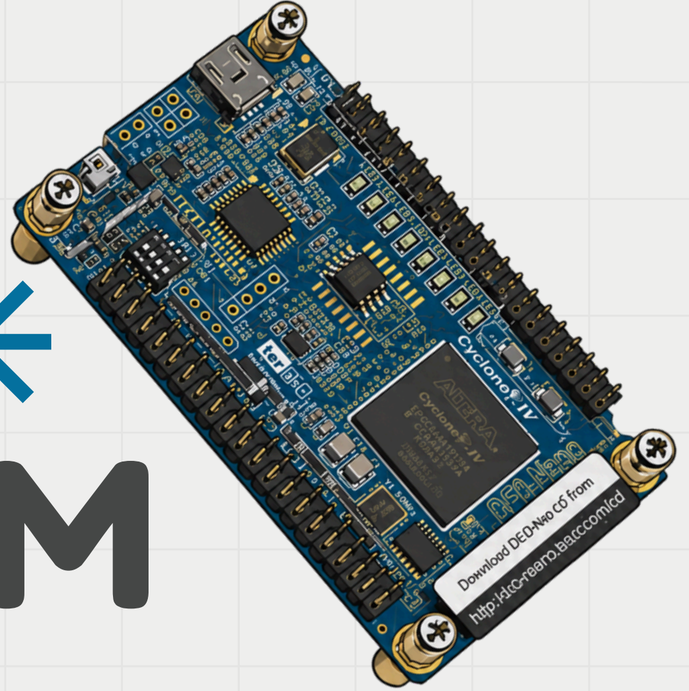


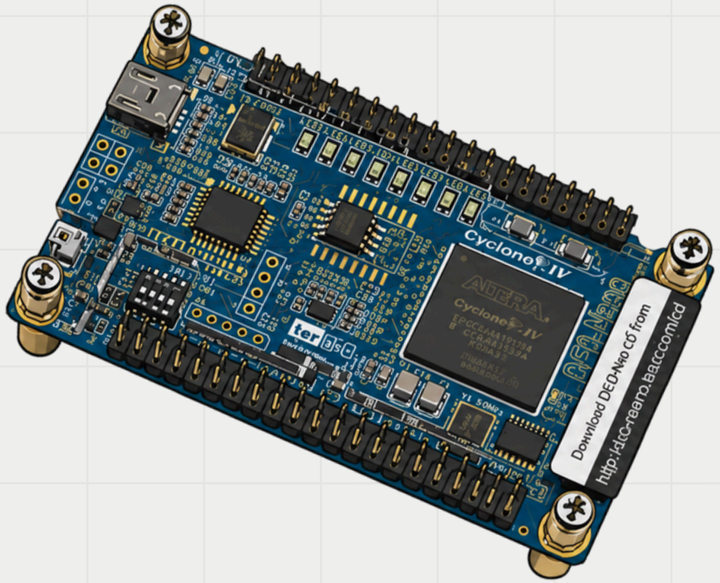


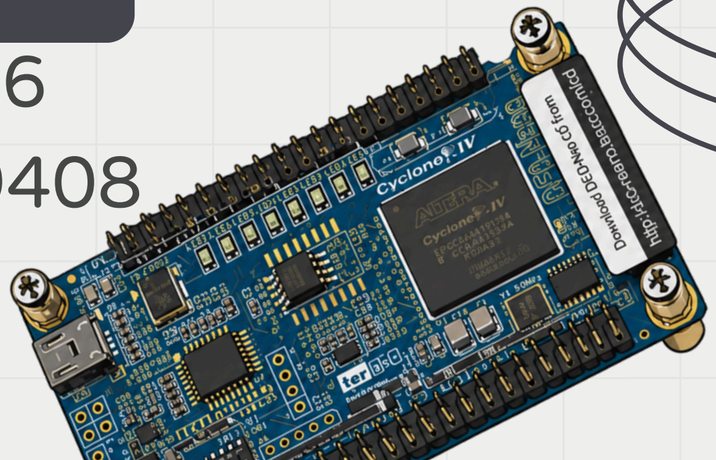
MEMÓRIA RAM 16X8 BITS

Simulada em VHDL e FPGA

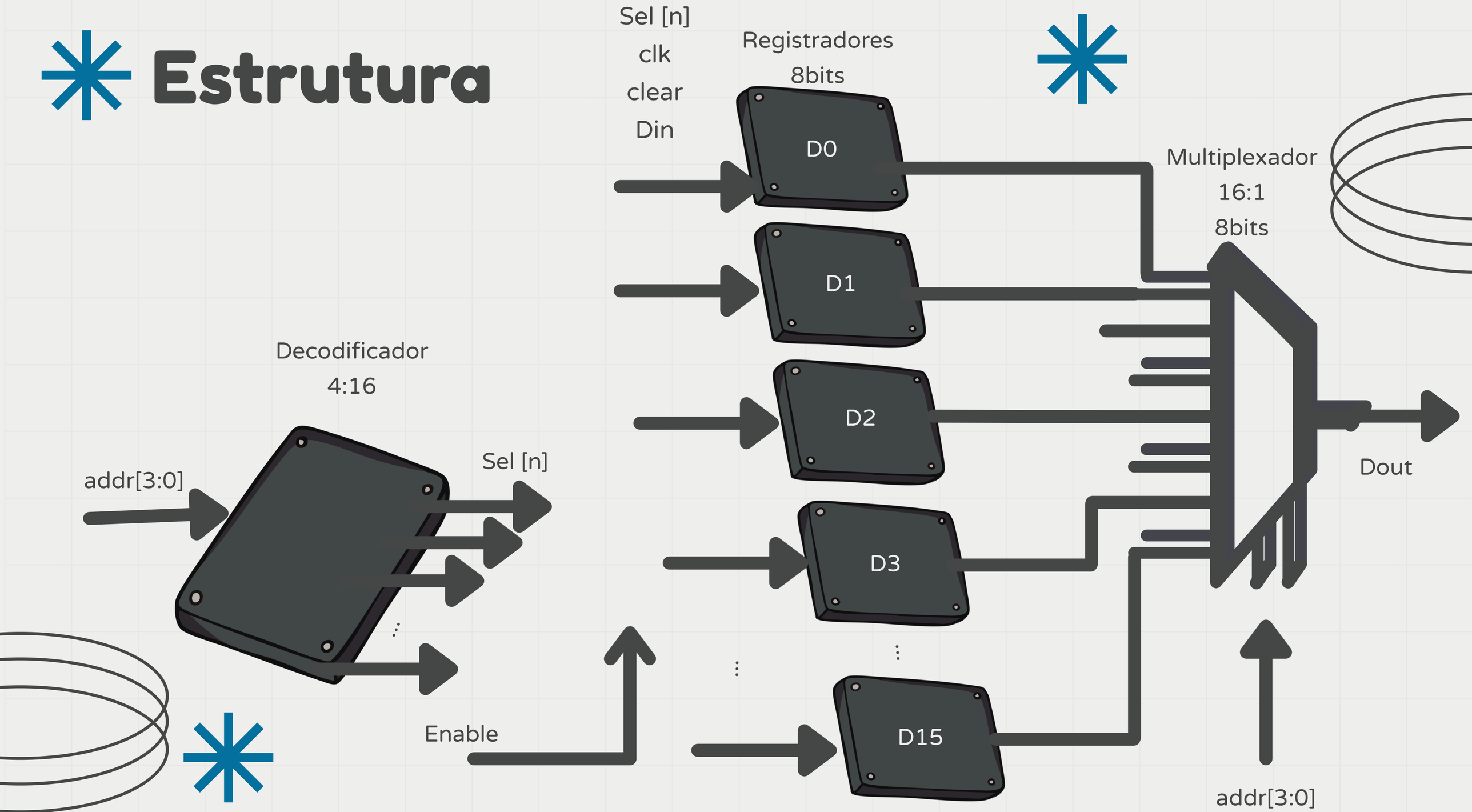
Letittja Velloso - 150886

Luiz Fernando Viana - 149408

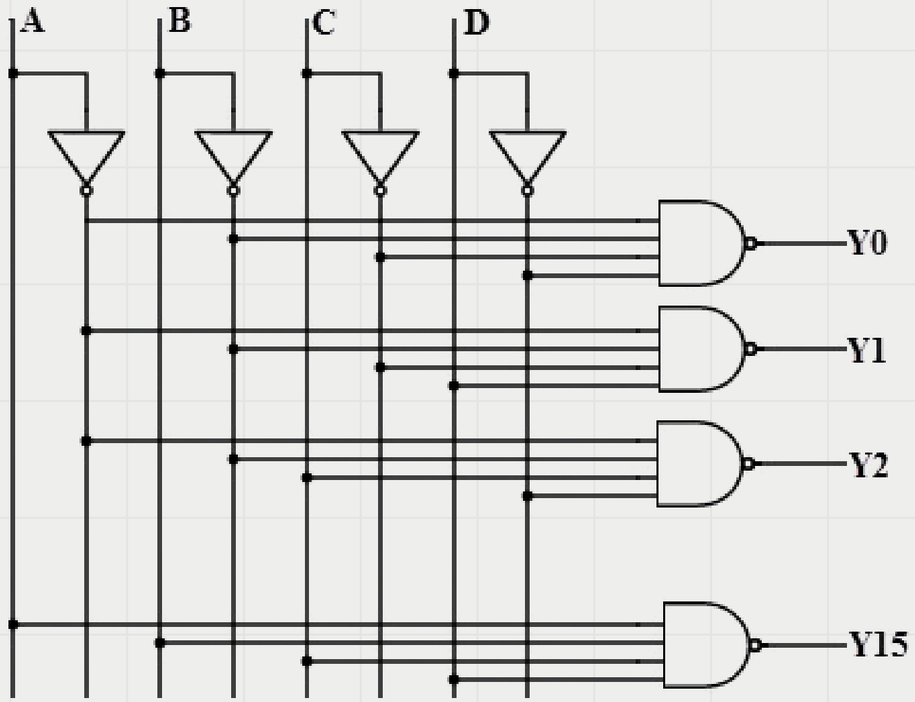




Estrutura

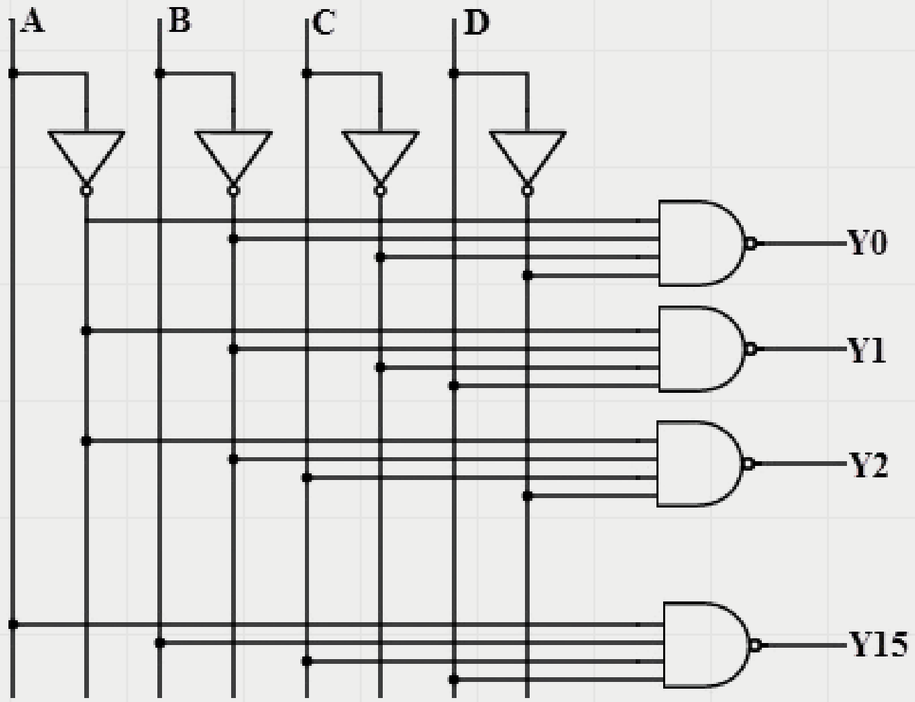

Sel [n]
clk
clear
Din
Registradores
8bits
D0
D1
D2
D3
D15
Multiplexador
16:1
8bits
Dout
Decodificador
4:16
addr[3:0]
Sel [n]
Enable
addr[3:0]

Decodificador 4:16



- Circuito lógico combinacional de 16 portas AND de 4 entradas cada, precedidas por inversores (portas NOT) para gerar as formas diretas e negadas de cada um dos 4 bits de entrada
- Recebe 4 bits de entrada e consegue formar 2^4 combinações binárias distintas;
- Cada combinação apenas uma das 16 linhas de saída é enviada para o nível lógico alto, enquanto as outras permanecem em nível lógico baixo.

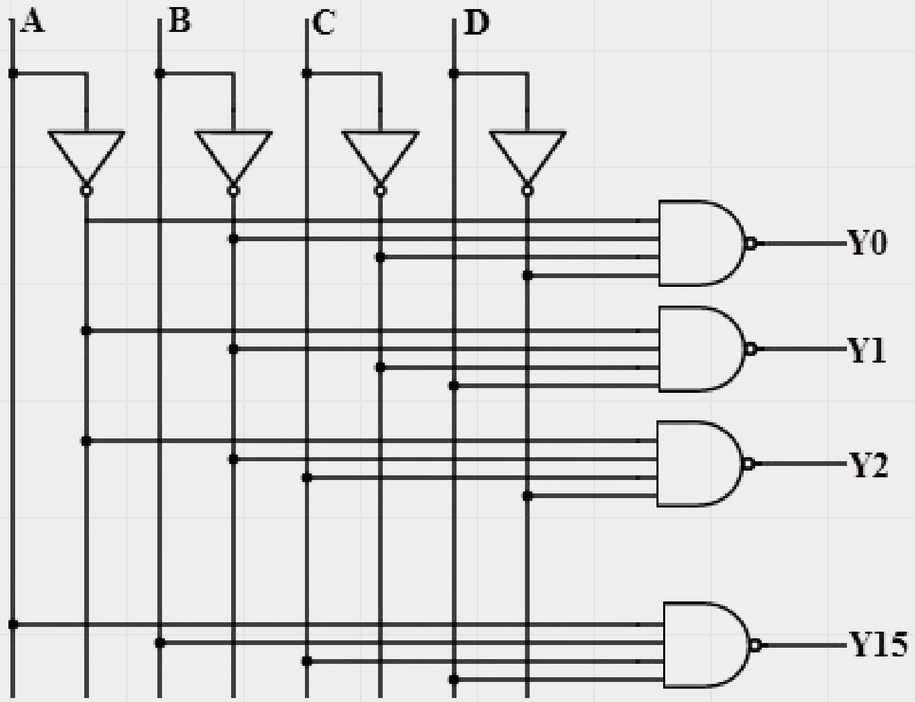
Decodificador 4:16



- Circuito lógico combinacional de 16 portas AND de 4 entradas cada, precedidas por inversores (portas NOT) para gerar as formas diretas e negadas de cada um dos 4 bits de entrada
- Recebe 4 bits de entrada e consegue formar 2^4 combinações binárias distintas;
- Cada combinação apenas uma das 16 linhas de saída é enviada para o nível lógico alto, enquanto as outras permanecem em nível lógico baixo.

- Exemplo: Se a entrada for 0000, a primeira saída (Y_0) é ativada. Se a entrada for 0001, a saída Y_1 é ativada, e assim por diante até a combinação 1111 que ativa a saída Y_{15}.

Decodificador 4:16



- Usado para seleção de endereços, escolhe uma posição específica na memória

- Circuito lógico combinacional de 16 portas AND de 4 entradas cada, precedidas por inversores (portas NOT) para gerar as formas diretas e negadas de cada um dos 4 bits de entrada
- Recebe 4 bits de entrada e consegue formar 2^4 combinações binárias distintas;
- Cada combinação apenas uma das 16 linhas de saída é enviada para o nível lógico alto, enquanto as outras permanecem em nível lógico baixo.

- Exemplo: Se a entrada for 0000, a primeira saída (Y_0) é ativada. Se a entrada for 0001, a saída Y_1 é ativada, e assim por diante até a combinação 1111 que ativa a saída Y_{15}.

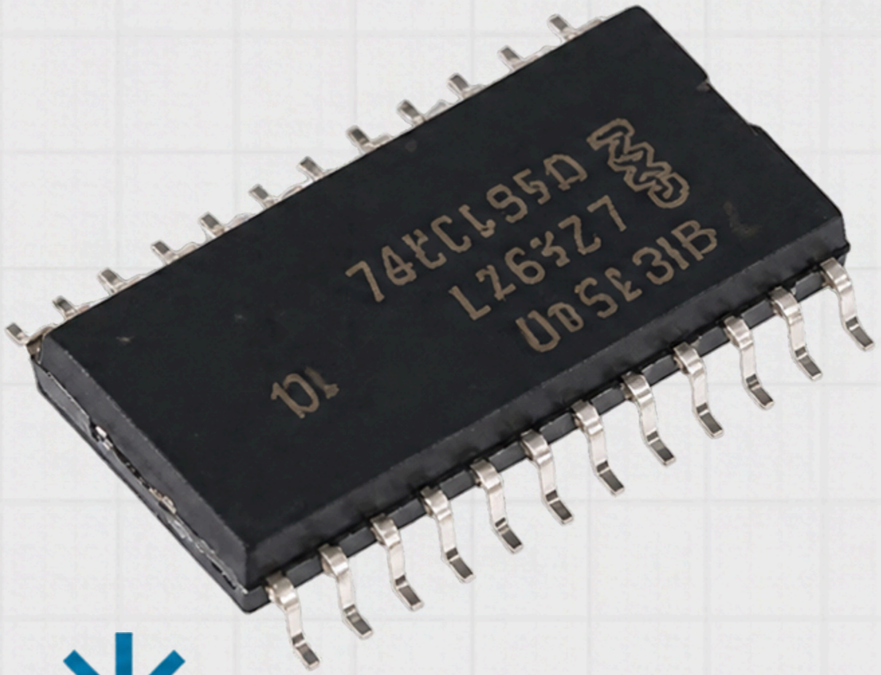
Como isso foi feito em VHDL?



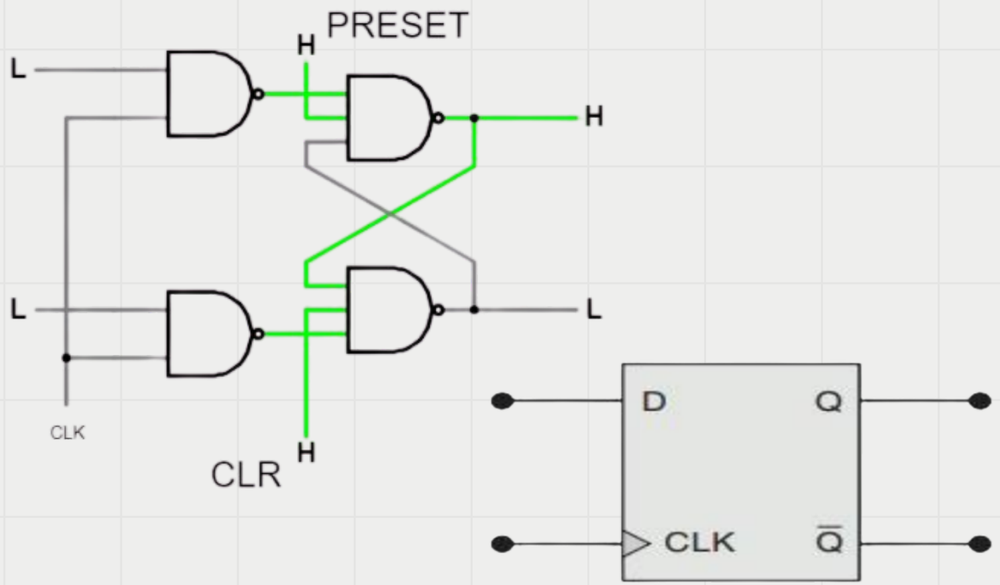
Como isso foi feito em VHDL?

```
entity decoder4to16 is
    port (
        addr : in  std_logic_vector(3 downto 0);
        sel  : out std_logic_vector(15 downto 0)
    );
end entity decoder4to16;

architecture rtl of decoder4to16 is
begin
    process (addr)
    begin
        sel <= (others => '0');
        sel(to_integer(unsigned(addr))) <= '1';
    end process;
end architecture rtl;
```

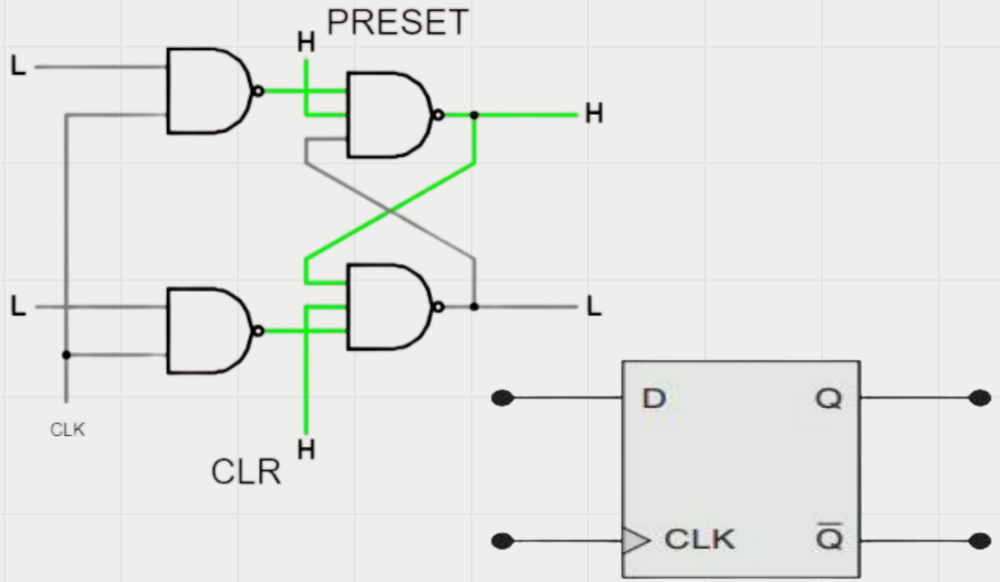


Registrador 16x8 Bits



- Circuito lógico composto por 8 flipflops D de 1 Bit cada, operando em paralelo;
- Sensível a borda de subida;
- Armazena e transfere um vetor de dados inteiro simultaneamente, atualizando seu valor apenas de forma síncrona (na borda do clock) e quando habilitado;
- Para cada registrador interno, a gravação de dados só é liberada mediante uma operação lógica AND entre a linha ativa vinda do decodificador de endereço e a presença do sinal enable.

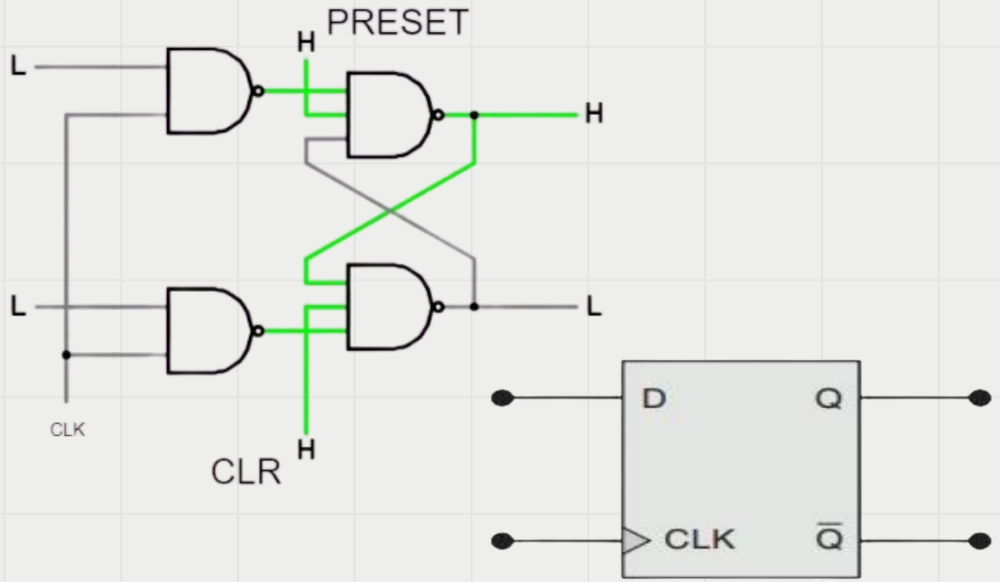
Registrador 16x8 Bits



- Circuito lógico composto por 8 flipflops D de 1 BIt cada, operando em paralelo;
- Sensível a borda de subida;
- Armazena e transfere um vetor de dados inteiro simultaneamente, atualizando seu valor apenas de forma síncrona (na borda do clock) e quando habilitado;
- Para cada registrador interno, a gravação de dados só é liberada mediante uma operação lógica AND entre a linha ativa vinda do decodificador de endereço e a presença do sinal enable.

- Exemplo: Se a entrada for 0000, ao aplicar uma entrada de dados e ocorre um pulso de clock, o valor presente na entrada (D) é transferido e armazenado na saída (Q) de cada flip-flop

Registrador 16x8 Bits



- Usado para armazenar de forma permanente (enquanto houver alimentação) uma palavra de 8 bits (um byte) em um único endereço dessa memória RAM.

- Circuito lógico composto por 8 flipflops D de 1 BIt cada, operando em paralelo;
- Sensível a borda de subida;
- Armazena e transfere um vetor de dados inteiro simultaneamente, atualizando seu valor apenas de forma síncrona (na borda do clock) e quando habilitado;
- Para cada registrador interno, a gravação de dados só é liberada mediante uma operação lógica AND entre a linha ativa vinda do decodificador de endereço e a presença do sinal enable.

- Exemplo: Se a entrada for 0000, ao aplicar uma entrada de dados e ocorre um pulso de clock, o valor presente na entrada (D) é transferido e armazenado na saída (Q) de cada flip-flop

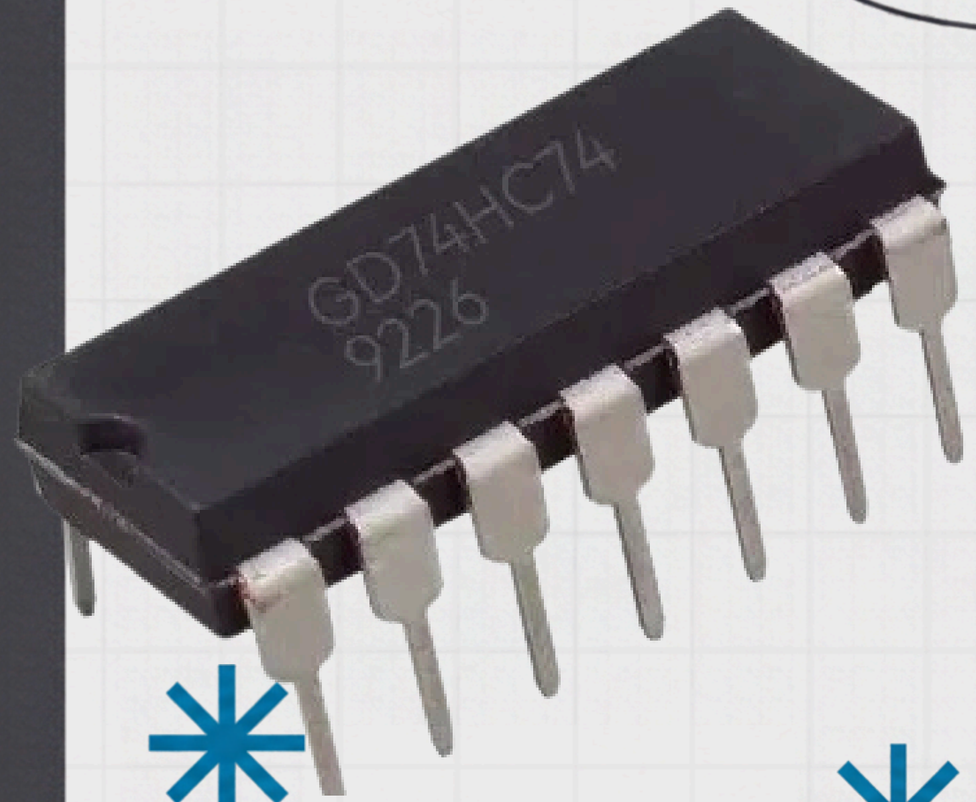
Como isso foi feito em VHDL?



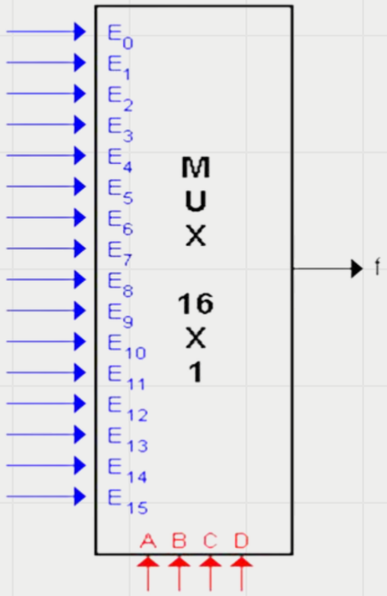
Como isso foi feito em VHDL?

```
entity reg8 is
    port (
        clk   : in  std_logic;
        clear : in  std_logic;
        en    : in  std_logic;
        d     : in  std_logic_vector(7 downto 0);
        q     : out std_logic_vector(7 downto 0)
    );
end entity reg8;
```

```
architecture rtl of reg8 is
begin
    process (clk, clear)
    begin
        if clear = '1' then
            q <= (others => '0');
        elsif rising_edge(clk) then
            if en = '1' then
                q <= d;
            end if;
        end if;
    end process;
end architecture rtl;
```

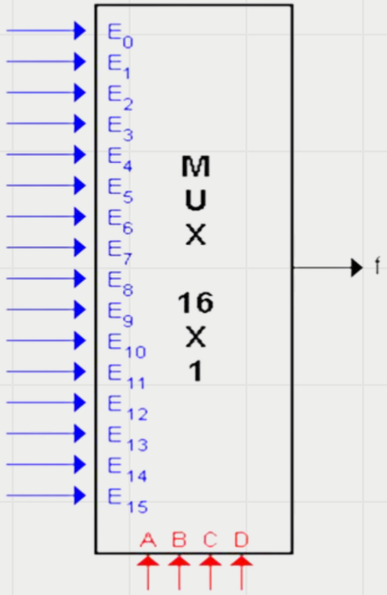


Multiplexador 16x1 8Bits



- Atua como uma chave seletora digital ou roteador;
- Utiliza um sinal de controle (endereço) para escolher qual das várias vias de entrada de dados terá seu sinal repassado para a saída;
- É essencial para o chaveamento de barramentos em sistemas digitais, direcionando blocos de dados de diferentes fontes para um único destino.

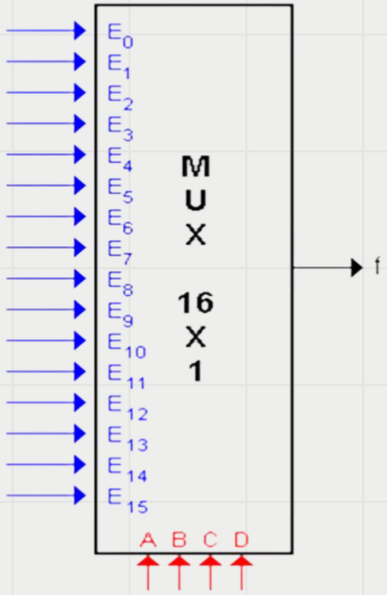
Multiplexador 16x1 8Bits



- Atua como uma chave seletora digital ou roteador;
- Utiliza um sinal de controle (endereço) para escolher qual das várias vias de entrada de dados terá seu sinal repassado para a saída;
- É essencial para o chaveamento de barramentos em sistemas digitais, direcionando blocos de dados de diferentes fontes para um único destino.

- Exemplo: Se a entrada em I_5 for 00000000, os pinos de Seleção ($S_3S_2S_1S_0$): 0101 (que é 5 em binário);
- Resultado: O MUX ativa o caminho interno de I_5 para a saída Y. Logo, Y = 00000000.

Multiplexador 16x1 8Bits

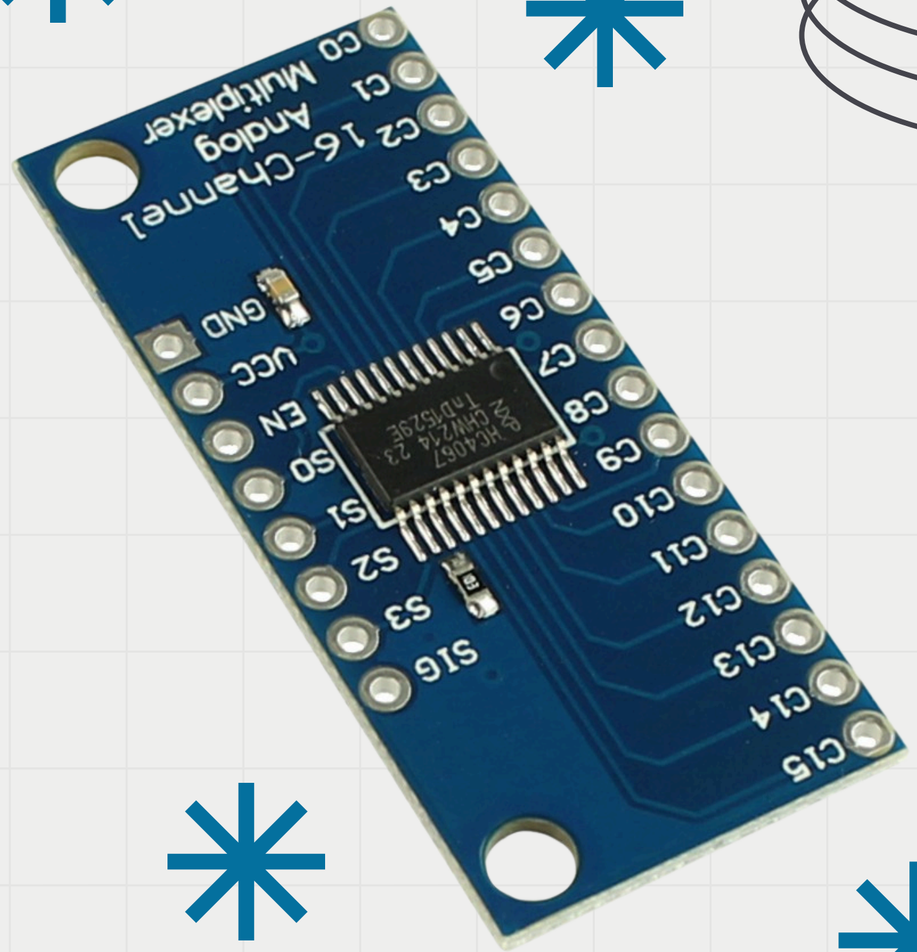


- O multiplexador usa 4 linhas de seleção ($2^4 = 16$ posições) oriundas do endereço de coluna da memória para decidir qual daquelas 16 opções é a correta.

- Atua como uma chave seletora digital;
- Utiliza um sinal de controle (endereço) para escolher qual das várias vias de entrada de dados terá seu sinal repassado para a saída;
- É essencial para o chaveamento de barramentos em sistemas digitais, direcionando blocos de dados de diferentes fontes para um único destino.

- Exemplo: Se a entrada em I_5 for 00000000, os pinos de Seleção ($S_3S_2S_1S_0$): 0101 (que é 5 em binário);
- Resultado: O MUX ativa o caminho interno de I_5 para a saída Y. Logo, Y = 00000000.

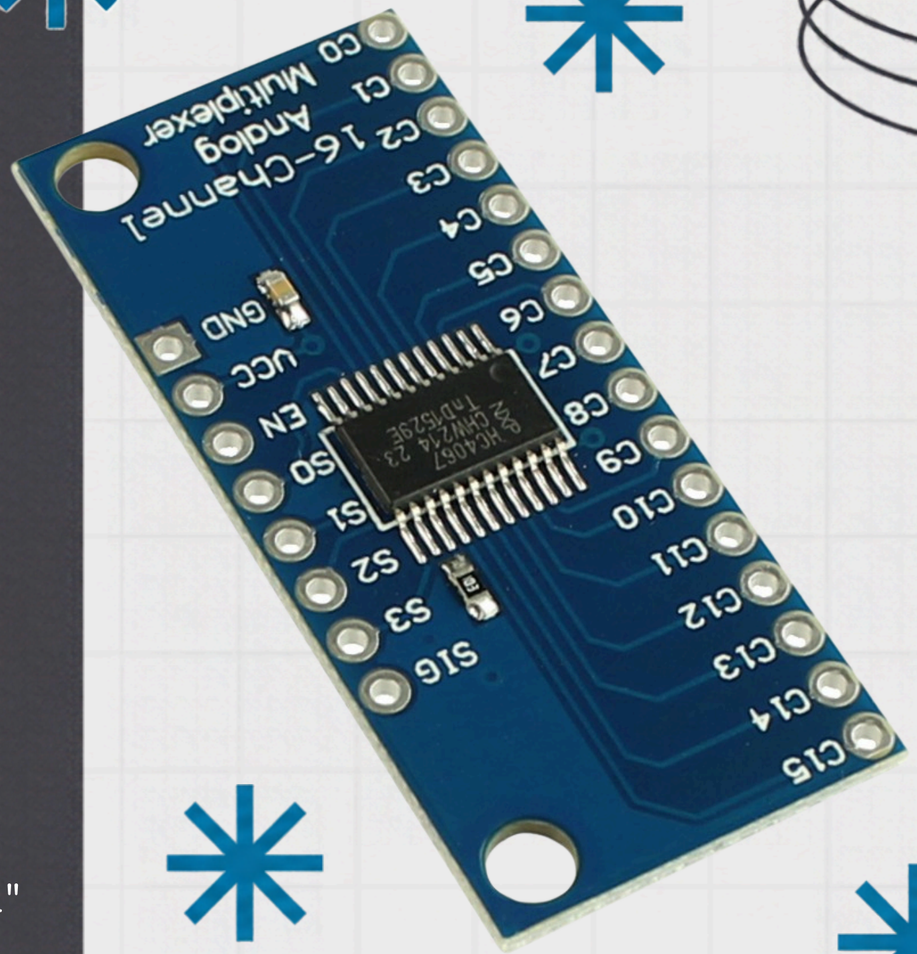
Como isso foi feito em VHDL?



Como isso foi feito em VHDL?

```
entity mux16x1_8bits is
    port (
        d0, d1, d2,  d3,  d4,  d5,  d6,
d7      :   in      std_logic_vector(7
downto 0);
         d8, d9, d10, d11, d12, d13,
d14,  d15 : in   std_logic_vector(7
downto 0);
         addr : in   std_logic_vector(3
downto 0);
        dout : out std_logic_vector(7
downto 0)
    );
end entity mux16x1_8bits;
```

```
architecture rtl of mux16x1_8bits is
begin
    with addr select
        dout <= d0  when "0000",
                d1  when "0001",
                d2  when "0010",
                d3  when "0011",
                d4  when "0100",
                d5  when "0101",
                d6  when "0110",
                d7  when "0111",
                d8  when "1000",
                d9  when "1001",
                        ...
                d15 when others;  -- "1111"
end architecture rtl;
```



Pin Planner

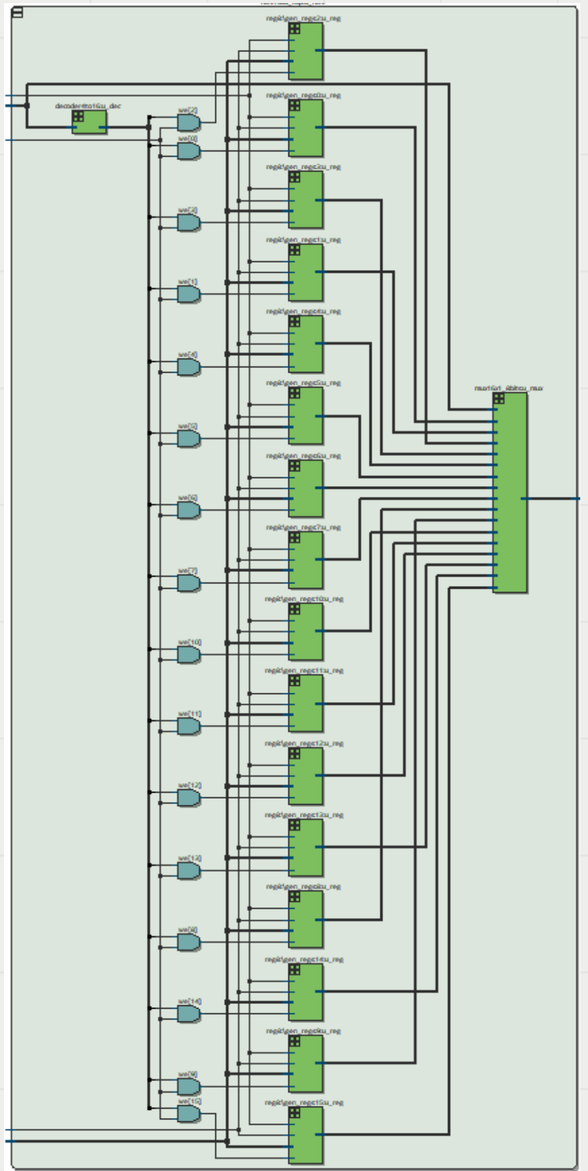
Node Name	Direction	Location	I/O Bank	VREF Group	itter Locatior	I/O Standard	Reserved	urrent Streng	Slew Rate	ifferential Pai	ict Preservati
addr[3]	Input	PIN_M15	5	B5_N0	PIN_A7	2.5 V ...fault)		8mA (default)			
addr[2]	Input	PIN_B9	7	B7_N0	PIN_B10	2.5 V ...fault)		8mA (default)			
addr[1]	Input	PIN_T8	3	B3_N0	PIN_B11	2.5 V ...fault)		8mA (default)			
addr[0]	Input	PIN_M1	2	B2_N0	PIN_M1	2.5 V ...fault)		8mA (default)			
clear	Input	PIN_E1	1	B1_N0	PIN_M2	2.5 V ...fault)		8mA (default)			
clk	Input	PIN_J15	5	B5_N0	PIN_E1	2.5 V ...fault)		8mA (default)			
D_in[7]	Input	PIN_B5	8	B8_N0	PIN_A11	2.5 V ...fault)		8mA (default)			
D_in[6]	Input	PIN_A4	8	B8_N0	PIN_A10	2.5 V ...fault)		8mA (default)			
D_in[5]	Input	PIN_B4	8	B8_N0	PIN_D9	2.5 V ...fault)		8mA (default)			
D_in[4]	Input	PIN_B3	8	B8_N0	PIN_F9	2.5 V ...fault)		8mA (default)			
D_in[3]	Input	PIN_A3	8	B8_N0	PIN_C9	2.5 V ...fault)		8mA (default)			
D_in[2]	Input	PIN_A2	8	B8_N0	PIN_C8	2.5 V ...fault)		8mA (default)			
D_in[1]	Input	PIN_C3	8	B8_N0	PIN_A15	2.5 V ...fault)		8mA (default)			
D_in[0]	Input	PIN_D3	8	B8_N0	PIN_E9	2.5 V ...fault)		8mA (default)			
D_out[7]	Output	PIN_L3	2	B2_N0	PIN_C11	2.5 V ...fault)		8mA (default)	2 (default)		
D_out[6]	Output	PIN_B1	1	B1_N0	PIN_C16	2.5 V ...fault)		8mA (default)	2 (default)		
D_out[5]	Output	PIN_F3	1	B1_N0	PIN_C6	2.5 V ...fault)		8mA (default)	2 (default)		
D_out[4]	Output	PIN_D1	1	B1_N0	PIN_E8	2.5 V ...fault)		8mA (default)	2 (default)		
D_out[3]	Output	PIN_A11	7	B7_N0	PIN_D8	2.5 V ...fault)		8mA (default)	2 (default)		
D_out[2]	Output	PIN_B13	7	B7_N0	PIN_N9	2.5 V ...fault)		8mA (default)	2 (default)		
D_out[1]	Output	PIN_A13	7	B7_N0	PIN_F8	2.5 V ...fault)		8mA (default)	2 (default)		
D_out[0]	Output	PIN_A15	7	B7_N0	PIN_E11	2.5 V ...fault)		8mA (default)	2 (default)		
enable	Input	PIN_A5	8	B8_N0	PIN_A12	2.5 V ...fault)		8mA (default)			
<<new node>>											

Pinagem DE0-Nano

Pinos na placa DNano0

Coluna	Fileira	Pino	Sinal
1	par	2	din[0]
2	par	4	din[1]
3	ímpar	5	din[2]
3	par	6	din[3]
4	ímpar	7	din[4]
4	par	8	din[5]
5	ímpar	9	din[6]
5	par	10	din[7]
7	ímpar	13	enable
15	ímpar	29	alimentação trilho de 3,3 V (+)
15	par	30	GND trilho de GND (-)

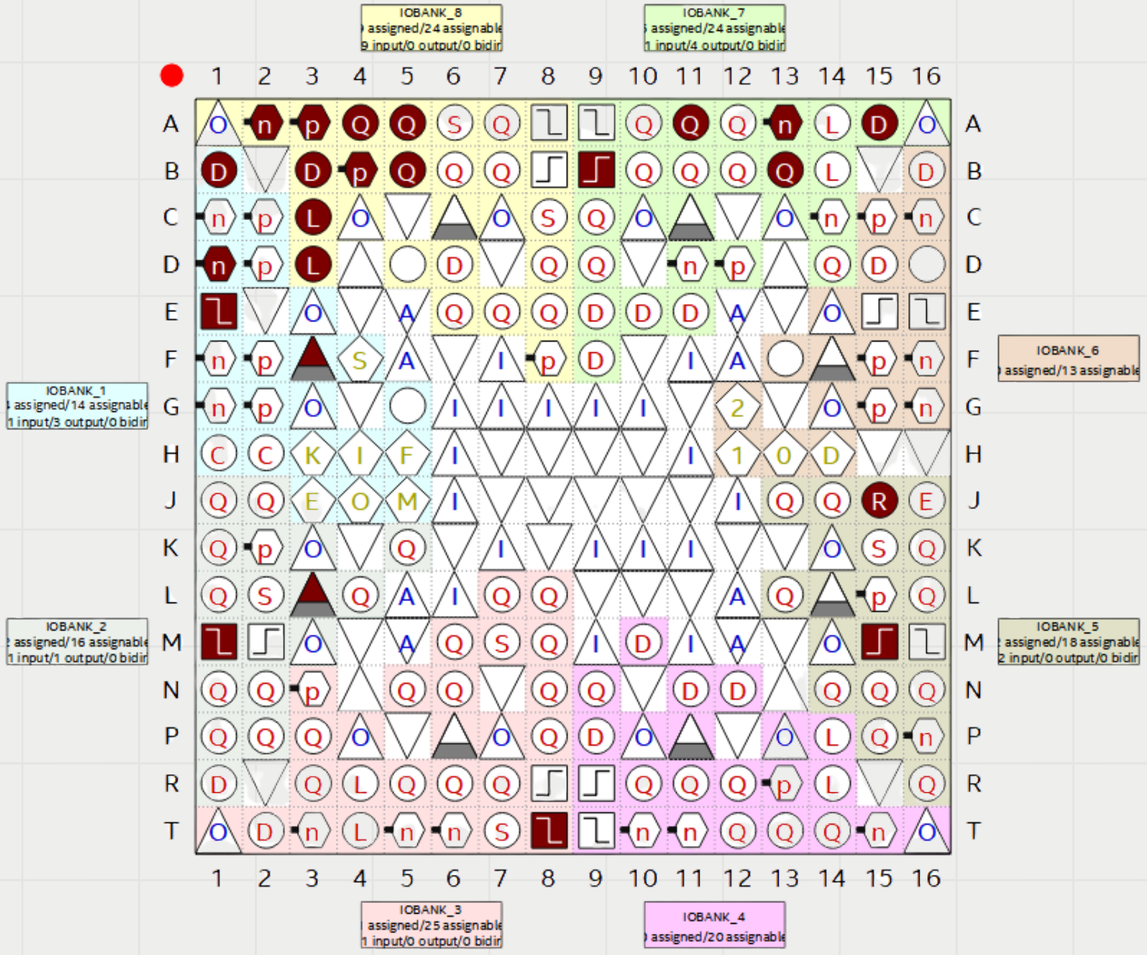
é da esquerda para a direita, de baixo para cima



Pinagem Chip

Top View - Wire Bond

Cyclone IV E - EP4CE22F17C6



Placa FPGA

